Séance 3

Écrire des fonctions avec R

BIO109 - Dominique Gravel

Laboratoire d'écologie intégrative | Integrative Ecology Lab [IE]

Retour sur l'exercice

Exercice de fin de séance

Le fichier quadrats.csv est un sommaire de données individuelles, où la présence de chaque espèce est mesurée. Ces données se trouvent dans arbres.csv. Dans le cadre de ce projet, on s'intéresse à la distribution de l'érable à sucre et des autres espèces tout au long du gradient d'élévation de la parcelle. Pour cet exercice, on vous demande de:

1. Charger les données "arbres"
2. Délimiter cinq zones au sein du gradient d'élévation : 0-200m, 201-400m, 401-600m, 601-800m, 801-1000m
3. Pour chacune de ces zones, calculer le nombre de tiges de chaque espèce
4. Enregistrer les résultats dans un tableau avec 5 rangées et S colonnes

On vous demande de rédiger un script qui réalisera l'ensemble de ces étapes, de la lecture des données à l'enregistrement du tableau final.

Solution

```r
# Créer un tableau où on enregistre les résultats
resultats <- matrix(nr = 5, nc = 7)
# Lire le fichier (en assumant que vous êtes dans le bon dossier)
arbres <- read.table(file="donnees/arbres.csv", header=TRUE, sep = ";")
# Délimiter une première zone
sub_zoneA <- subset(arbres, arbres$bory < 201)
# Calculer le nombre de tiges
table(sub_zoneA$esp)
```

```
##
## abba acpe acsa beal bepa fagr piru
##   70 1038 1140  587   51 2456  227
```

```r
# Enregistrer le résultat dans le tableau
resultats[1,] <- table(arbres$esp)
```

Fonction source

Il est possible d'exécuter un script sans avoir à l'ouvrir.

```
source("MonScript.R")
```

Opérations mathématiques de base

Astuce: générer des nombres aléatoires

Il peut souvent être pratique de générer des chiffres au hasard, sur lesquels on souhaite faire des tests. Nous verrons plusieurs exemples au cours 5, mais pour l'instant, prenez note de la fonction suivante:

```
alea ← runif(n = 10, min = 0, max = 1)
alea
```

```
##  [1] 0.05033141 0.66171378 0.63318621 0.81133016 0.57868468 0.32688816
##  [7] 0.20761250 0.22439947 0.93536759 0.27401614
```

Ici la distribution utilisée est la distribution uniforme, toutes les autres distributions en sont dérivées.

Opérateurs de base

R peut faire toutes les opérations mathématiques de base d'une calculatrice :

Opérateur	Signification	Exemple
+	Addition	3 + 7
-	Soustraction	3 - 7
*	Multiplication	3 * 7
/	Division	3 / 7

Opérations sur des vecteurs et matrices

Par défaut, le produit de vecteurs et de matrices est *scalaire* :

```
vec1 ← runif(10, min = 0, max = 1)
vec1
```

```
##  [1] 0.27781612 0.67770481 0.54999326 0.38462129 0.43165191 0.99432674
##  [7] 0.26173136 0.02570071 0.57739793 0.66105437
```

```
3 * vec1
```

```
##  [1] 0.83344837 2.03311442 1.64997977 1.15386386 1.29495574 2.98298023
##  [7] 0.78519408 0.07710213 1.73219379 1.98316312
```

Opérations sur des vecteurs et matrices

La situation plus compliquée survient lorsque l'on multiplie des vecteurs et des matrices :

```
vec1 ← c(10,20,30)
mat1 ← matrix(c(1:6), nr = 3, nc = 2)
mat1
```

```
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
```

```
vec1 * mat1
```

```
##      [,1] [,2]
## [1,]   10   40
## [2,]   40  100
## [3,]   90  180
```

Opérations avancées

Opérateur	Signification	Exemple
^	Puissance	3 ^ 7
log	Logarithme	log(100)
log10	Logarithme à base 10	log10(100)
exp	Exponentielle	exp(10)
sqrt	Racine carrée	sqrt(100)

Opérations avancées

```
alea ← runif(10, min = 0, max = 1)
alea
```

```
##  [1] 0.3610274 0.2606482 0.4979184 0.8260666 0.8382290 0.5097082 0.8860412
##  [8] 0.4144201 0.3081929 0.6523749
```

```
min(alea)
```

```
## [1] 0.2606482
```

```
max(alea)
```

```
## [1] 0.8860412
```

Arrondir

```
pi
```

```
## [1] 3.141593
```

```
floor(pi)
```

```
## [1] 3
```

```
ceiling(pi)
```

```
## [1] 4
```

```
round(pi,4)
```

```
## [1] 3.1416
```

Opérations sur des matrices

Parfois, on souhaite calculer des propriétés sur des colonnes et des rangées:

```
mat ← matrix(runif(4,0,1), nr = 2, nc = 2)
mat
```

```
##            [,1]      [,2]
## [1,] 0.15733080 0.6550306
## [2,] 0.01431704 0.2150666
```

```
rowSums(mat)
```

```
## [1] 0.8123614 0.2293836
```

```
colSums(mat)
```

```
## [1] 0.1716478 0.8700972
```

Opérations sur des matrices

Et de façon plus générale, on peut utiliser la fonction `apply()` qui est très pratique :

```
mat ← matrix(1:4, nr = 2, nc = 2)
mat
```

```
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
```

```
apply(X = mat, MARGIN = 2, FUN = sum)
```

```
## [1] 3 7
```

```
apply(X = mat, MARGIN = 1, FUN = sum)
```

```
## [1] 4 6
```

Exercice

Abondances relatives

Au moyen du fichier quadrats.csv, je vous demande de faire les opérations suivantes:

1. Calculer l'abondance totale pour 'abba' et son abondance moyenne ;
2. Calculer le nombre total de tiges pour chaque quadrat ;
3. Calculer l'abondance relative pour 'abba' pour chaque quadrat. C-a-d le nombre de tiges de 'abba' divisé par le nombre total de tiges, pour chaque quadrat.

```
##       X abba acpe acsa beal bepa fagr piru
## 1   0-0    1   55   11    7    0   92    0
## 2 0-100    0    5    4    3    0    6    0
## 3 0-120    2    7   12    4    1    7    0
## 4 0-140    4    5    4    8    1    2    1
## 5 0-160    2    2   11    8    1    6    1
## 6 0-180    5    3    9    7    0    3    1
```

Abondances relatives

```r
# 0. Charger les données
quadrats <- read.table(file="donnees/quadrats.csv", header=TRUE, sep=";", dec=".")

# 1.1 Calculer l'abondance totale pour 'abba'
n_total_abba <- sum(quadrats$abba)
n_total_abba
```

```
## [1] 2596
```

```r
# 1.2 Calculer l'abondance moyenne pour 'abba'
n_moy_abba <- mean(quadrats$abba)
n_moy_abba
```

```
## [1] 5.202405
```

Abondances relatives

```r
# 2. Calculer le nombre total de tiges pour chaque quadrat
n_quadrats <- rowSums(quadrats[, 2:8])

# 3. Calculer l'abondance relative pour 'abba' pour chaque quadrat
quadrats_rel_abba <- quadrats$abba / n_quadrats
head(quadrats_rel_abba)
```

```
## [1] 0.006024096 0.000000000 0.060606061 0.160000000 0.064516129 0.178571429
```

Les fonctions

Qu'est-ce qu'une fonction ?

Une fonction contient une série de commandes (i.e. lignes de code) qui sont exécutées lorsque la fonction est appelée.

Anatomie de la fonction

```
ma_fonction ← function(argument1, argument2) {

  # Ce que l'on veut que la fonction exécute
  resultat ← argument1 * argument2

  # Retourner le résultat de la fonction
  return(resultat)
}
```

Anatomie de la fonction

```r
ma_fonction <- function(argument1, argument2) { <---- Nom de la fonction et argument

  # Ce que l'on veut que la fonction exécute
  resultat <- argument1 * argument2             <---- Le corps de la fonction

  # Retourner le résultat de la fonction
  return(resultat)                              <---- La valeur retournée
}
```

```r
ma_fonction(2, 5)
```

```
## [1] 10
```

Pourquoi utiliser des fonctions ?

1. Répéter une même tâche mais en changeant ses paramètres
2. Rendre votre code plus lisible
3. Rendre votre code plus facile à modifier et à maintenir
4. Partager du code entre différentes analyses
5. Partager votre code avec d'autres personnes
6. Modifier les fonctionalités par défaut de R

Pourquoi utiliser des fonctions ?

Vous connaissez déjà des fonctions de R, comme `mean()` ou `sum()` !

```
vecteur ← c(1, 2, 3, 4, 5)
mean(vecteur)
```

```
## [1] 3
```

```
sum(vecteur)
```

```
## [1] 15
```

La construction d'une fonction

Imaginons que l'on souhaite multiplier deux chiffres (disons, 3 et 7) et les diviser par leur somme.

$$\frac{3 \times 7}{3 + 7}$$

On peut écrire ce calcul directement dans la console comme suit

```
(3*7)/(3+7)
```

```
## [1] 2.1
```

La construction d'une fonction

Si on souhaite faire la même opération pour toutes les paires de chiffres dans le tableau suivant, comment fait-on ?

```
tableau ← data.frame(x=rnorm(5), y=rnorm(5))
tableau
```

```
##            x          y
## 1  0.2707500  0.2560612
## 2  0.1728761 -2.5427911
## 3  0.5215657  0.7232796
## 4 -0.4310835 -1.3960053
## 5  0.6508865  0.9824026
```

La construction d'une fonction

À noter qu'en ayant différents chiffres, la formule vue dans la diapositive précédente devient un peu plus générale :

$$\frac{x \times y}{x + y}$$

La construction d'une fonction

L'approche longue, pas efficace, mais qui marche...

```
(tableau[1,1]*tableau[1,2])/(tableau[1,1]+tableau[1,2])
(tableau[2,1]*tableau[2,2])/(tableau[2,1]+tableau[2,2])
(tableau[3,1]*tableau[3,2])/(tableau[3,1]+tableau[3,2])
```

```
## [1] 0.1316004
## [1] 0.1854867
## [1] 0.3030399
```

Problème - Ce n'est vraiment pas pratique si on a beaucoup de données ou si le format du tableau change.

La construction d'une fonction

Une boucle peut sauver du temps...

```
for(i in 1:nrow(tableau)){
  res ← (tableau[i,1]*tableau[i,2])/(tableau[i,1]+tableau[i,2])
  print(res)
}
```

```
## [1] 0.1316004
## [1] 0.1854867
## [1] 0.3030399
## [1] -0.3293736
## [1] 0.3915
```

Problème - Qu'est-ce qu'on fait si on veut appliquer ce calcul sur plusieurs tableaux ??? *C'est possible, mais ça peut être un peu plus compliqué !*

La construction d'une fonction

Et si on faisait une fonction...

La fonction permet de résoudre certains problèmes car elle permet d'appliquer une série de commandes (i.e. lignes de codes) à différents types de données. En d'autre mots, la fonction généralise des commandes spécifiques.

La construction d'une fonction

Comment construire une fonction ?

On commence par écrire une version spécifique du code que l'on souhaite **généraliser**.

```
(3*7)/(3+7)
```

La construction d'une fonction

Comment construire une fonction ?

Ensuite, on définit ce code comme faisant parti d'une fonction.

À vous de l'essayer !

La construction d'une fonction

Comment construire une fonction ?

Ensuite, on définit ce code comme faisant parti d'une fonction.

```
prodsum ← function(){
  res ← (3*7)/(3+7)
  return(res)
}
```

Yééé, on a écrit notre première fonction !! :-)

```
prodsum()
```

```
## [1] 2.1
```

Petit problème

Cette fonction a le défaut de n'être aucunement générale. Elle va toujours donner le même résultat. :-(

Comment rendre la fonction plus générale ?

On peut généraliser cette fonction, en utilisant directement la formule générale présentée précédemment:

$$\frac{x \times y}{x + y}$$

Pour ce faire, il faut ajouter des **arguments** à notre fonction.

À vous de l'essayer !

Comment rendre la fonction plus générale ?

Les **arguments** peuvent varier selon ce que l'utilisateur souhaite calculer. Il faut donc s'assurer que les mêmes opérations soient réalisées sur ces arguments.

```
prodsum = function(x,y){
  res ← (x*y)/(x+y)
  return(res)
}
```

Avec cette fonction on peut faire le calcul qui nous intéresse avec différentes séries de chiffres.

Comment construire une fonction ?

À noter qu'en utilisant la commande `return()`, on s'assure de renvoyer ce qui se trouve dans l'objet `res` à l'utilisateur.

Une notion importante à avoir lorsqu'on construit une fonction est que tout ce qui se trouve à l'**intérieur** d'une fonction est *entièrement* indépendent de ce qui se trouve à l'**extérieur** d'une fonction.

Par exemple, l'objet `res` à l'extérieur de la fonction **n'existe pas**. Il existe uniquement à l'intérieur de la fonction.

Comment rendre la fonction plus générale ?

Autre charactéristique importante des arguments d'une fonction: les objets passés en argument n'ont pas besoin d'avoir le même nom que les arguments. En fait, c'est rarement le cas :

```
a ← 1
b ← 2
prodsum(x = a, y = b)
```

```
## [1] 0.6666667
```

Utilisons notre fonction avec des chiffres

```
prodsum(3, 7)
```

```
## [1] 2.1
```

```
prodsum(y = 7, x = 3)
```

```
## [1] 2.1
```

Comme on le constate, dans le language R, les arguments peuvent être définis de deux façons.

- En suivant l'ordre des arguments
- En utilisant le noms des arguments

Utilisons notre fonction avec des vecteurs

Le language de programmation R permet de faire aussi le calcul sur des vecteurs :

```
vecA = tableau[,1]
vecB = tableau[,2]
prodsum(vecA, vecB)
```

```
## [1]  0.1316004  0.1854867  0.3030399 -0.3293736  0.3915000
```

```
prodsum(tableau[,1], tableau[,2])
```

```
## [1]  0.1316004  0.1854867  0.3030399 -0.3293736  0.3915000
```

Utilisons notre fonction avec des vecteurs

L'exemple précédent fonctionne bien car `vecA` et `vecB` contiennent le même nombre de chiffres, ils ont la même longueur. Que ce passe-t-il si les vecteurs n'ont pas la même longueur?

```
vec2 = tableau[1:2,1]
vec3 = tableau[1:3,1]
vec4 = tableau[1:4,2]
prodsum(vec3, vec4)
```

```
## Warning in x * y: longer object length is not a multiple of shorter object
## length

## Warning in x + y: longer object length is not a multiple of shorter object
## length

## [1] 0.1316004 0.1854867 0.3030399 0.3358958
```

La construction d'une fonction

Le **recyclage** est une propriété des fonctions mathematiques de base du language R (e.g. `+`, `-`, `*` et `/`). Lorsque deux vecteurs sont de longueurs différentes, les valeurs du vecteur le plus court sont réutilisées, dans l'ordre, pour combler le nombre de valeurs manquantes entre les deux vecteurs. Cette propriété du langage R peut être très pratique mais aussi **générer beaucoup de problèmes**.

Comme on le voit dans l'exemple, un message d'avertissement est envoyé si la longueur du plus petit vecteur n'est pas un multiple du vecteur le plus long. Par contre, si le vecteur le plus court est un multiple du vecteur le plus long, aucun message d'avertissement n'est envoyé et ce même si le résultat n'a pas de sense.

Comment faire pour régler ce problème ?

Ajouter des trappes dans une fonction

Pour régler le problème subtile présenté dans la diapositive précédente, on peut ajouter des conditions de sorties dans la fonction qui retournent un message d'erreur. Voici un exemple :

```
prodsum = function(x, y){
  if(length(x) ≠ length(y)){
    stop("'x' est de taille différente de 'y'")
  }
  res = (x*y)/(x+y)
  return(res)
}
```

Les lignes de codes ajoutées mesure la longeur de `x` et `y` et lorsqu'ils sont d'une longueur différente, un message d'erreur est envoyé et le reste du code dans la fonction n'est pas évalué.

Ajouter des trappes dans une fonction

La condition de sortie `stop("'x' est de taille différente de 'y'")` permet de corriger le problème mentionné dans la diapositive précédente :

```
prodsum(vec2, vec4)
```

```
## Error in prodsum(vec2, vec4): 'x' est de taille différente de 'y'
```

```
prodsum(vec3, vec4)
```

```
## Error in prodsum(vec3, vec4): 'x' est de taille différente de 'y'
```

Notez que le message d'erreur envoyé est composé par le programmeur. Par contre, il est important que le message d'erreur soit court et précis.

Utilisons notre fonction avec des tableaux

Le language de programmation R permet de faire aussi le calcul sur des matrices

```
tableau2 ← data.frame(x_2 = rnorm(5)^2, y_2 = rnorm(5)^2)
prodsum(tableau, tableau2)
```

```
##             x            y
## 1 0.201459820 0.0923643613
## 2 0.159678058 1.7712442735
## 3 0.004939666 0.0008434981
## 4 0.032615005 0.2602959895
## 5 0.185207024 0.1557918009
```

La sortie

Une fonction peut réaliser plusieurs opérations avec les mêmes éléments d'entrée et on peut souhaiter retourner ces arguments dans une liste :

```
ma_fonction = function(x) {

  # Calcul de la moyenne
  res1 ← mean(x)

  # Calcul de l'écart-type
  res2 ← sd(x)

  # On regroupe les résultats dans une liste
  final ← list(moyenne = res1, ecart_type = res2)

  # Et on retourne le tout hors de la fonction
  return(final)
}
```

La sortie

Une fonction peut réaliser plusieurs opérations avec les mêmes éléments d'entrée et on peut souhaiter retourner ces arguments dans une liste :

```
test ← ma_fonction(rnorm(n = 100, mean = 1, sd = 0.5))
test
```

```
## $moyenne
## [1] 0.9997225
##
## $ecart_type
## [1] 0.5415556
```

Quelques trucs utiles

- Prenez le temps d'écrire votre code pour un exemple avant de tout intégrer dans une fonction
- Tout ce qui se trouve à l'intérieur d'une fonction est **entièrement** indépendent de ce qui se trouve à l'extérieur de cette fonction ;
- Donner une nom représentatif à la fonction, qui résume ce qu'elle fait ;
- Tous les arguments de la fonction doivent être utilisés;
- Les arguments doivent aussi avoir des noms représentatifs ;
- Commentez les étapes du code ;
- Prenez le temps de décrire les arguments sous forme de commentaires en début de fonction ;

Exercice de fin de séance

Abondances relatives

Toujours à partir du fichier quadrats.csv, programmez une fonction qui retournera les statistiques descriptives suivantes pour une espèce :

1. L'abondance moyenne
2. L'abondance totale
3. Le coefficent de variation de l'abondance
4. L'abondance maximale dans un quadrat
5. L'abondance minimale dans un quadrat

Ensuite, appliquez cette fonction sur chacune des espèces (colonnes)

Rappel : identifiez bien dès le départ l'objet que vous passez comme argument de votre fonction.